

Mutex Tests Timing

Each of 3 thread will go through a loop 10 million times incrementing a shared variable. Please read the results and then the conclusion below them.

Mutex Protected - that is incrementing of the variable is being protected by a mutex.

Test 1	Test 2	Test3
real 0m1.848s	real 0m1.700s	real 0m1.746s
user 0m3.735s	user 0m3.226s	user 0m3.497s
sys 0m1.586s	sys 0m1.602s	sys 0m1.585s

No Mutexes - this is the previous test with the mutexes commented out.

Test 1	Test 2	Test 3
real 0m0.089s	real 0m0.090s	real 0m0.091s
user 0m0.259s	user 0m0.260s	user 0m0.261s
sys 0m0.000s	sys 0m0.004s	sys 0m0.004s

No Mutexes and one thread at a time.

Test 1	Test2	Test 3
real 0m0.081s	real 0m0.095s	real 0m0.094s
user 0m0.077s	user 0m0.095s	user 0m0.095s
sys 0m0.004s	sys 0m0.000s	sys 0m0.000s

Spin Lock

Test 1	Test 2	Test 3
real 0m19.657s	real 0m13.004s	real 0m14.011s
user 0m58.346s	user 0m36.991s	user 0m35.978s
sys 0m0.004s	sys 0m0.024s	sys 0m0.004s

The following are some conclusions that we can make from these results. Keep in mind that this is a simple usage with a lot of lock and unlock calls.

- Using mutexes can be necessary, but they do incur a cost. We are seeing a factor of more than 10 speed reduction when we use mutexes for a simple task. Happily, this is not a realistic test. Why?
- Using threads is not always beneficial. Consider the performance improvement in using one thread at a time over multiple threads at the same time. This is due to data caching on the processor chip. The caching limits processors accessing the same memory locations.
- Spin locks are not necessarily better than mutexes. It depends on how often they are locked when another thread needs access.